Group 8

人工智慧概論*project*

組員: 吳俊良、伍文安、陳柏諺、林哲宇、呂佳芸

Introduction

Problem Formulation

Among the problems that LLMs can’t solve well, some of them are problems/tasks requiring fast response and/or precise control, such as controlling machines, driving vehicles, or playing games.

Often there are other AI methods that are more suitable in solving these problems, designed specifically to handle them and other similar tasks.

Problem

We then selected car driving/racing as the problem for our project, as this problem requires both fast response and precise control for continuous control.

Problem environment: OpenAI Gym: CarRacing-v3



Proposed Method

Soft Actor Critic (SAC)

An off-policy deep reinforcement learning algorithm designed for continuous control tasks. Designed to keep performing stable exploration through maximum entropy while also trying to get the best reward. It can learn from old data stored in buffer, which makes it an off-policy method.

Why SAC?

Despite no longer being a SoTA RL method for continuous control tasks, SAC is still one of the best RL methods. With its stable learning and exploration, and efficient off-policy sample efficiency, it’s still capable of producing a robust policy for our problem/task.

What if LLM is used instead?

With LLMs, response often takes time that are slower compared to low-level RL methods designed for specific tasks/problems. This is apparent in problems or tasks requiring fine-grained continuous control, as LLM methods performs poorly compared to RL methods [1].

Car driving/racing are one example of problems requiring fine-grained continuous control, and thus LLM methods would also not perform well in this problem.

1. Hatgis-Kessell, S., & Brunskill, E. (2026, May 29). When are LLMs Sufficient Policy Optimizers for Sequential RL Tasks? arXiv.org. https://arxiv.org/abs/2605.30719

Model Design

version 0

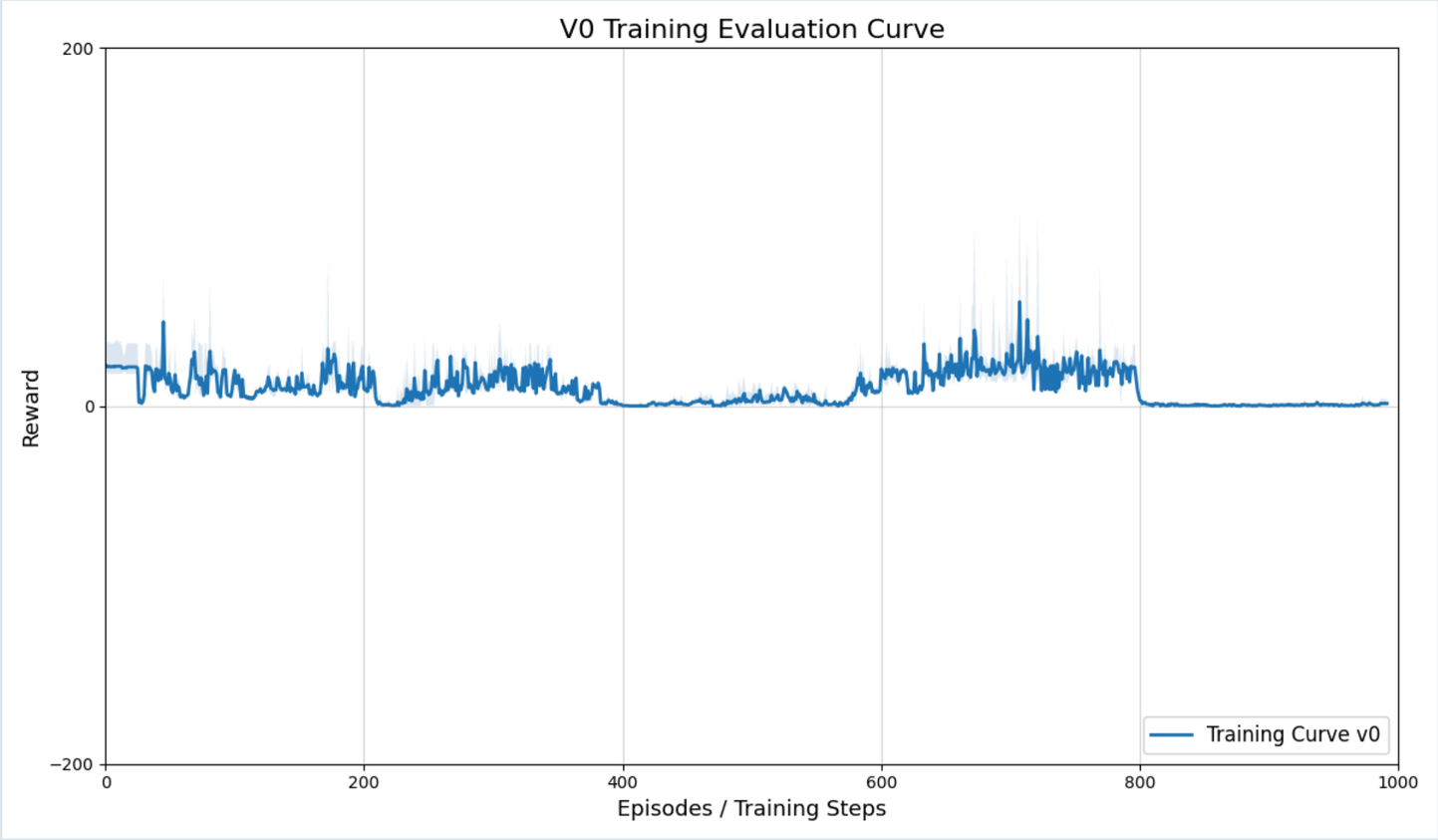
version 0: vanilla baseline that allows the car to drive unconstrained

Input: Track image + dashboard status

Model: Actor + Double Critic

Training: Replay Buffer + checkpoint

Evaluation: Every 1,000 steps



version 1

version 1: stuck-preventing, outside-preventing environment

Difference from V0:

- **Stuck Detection**

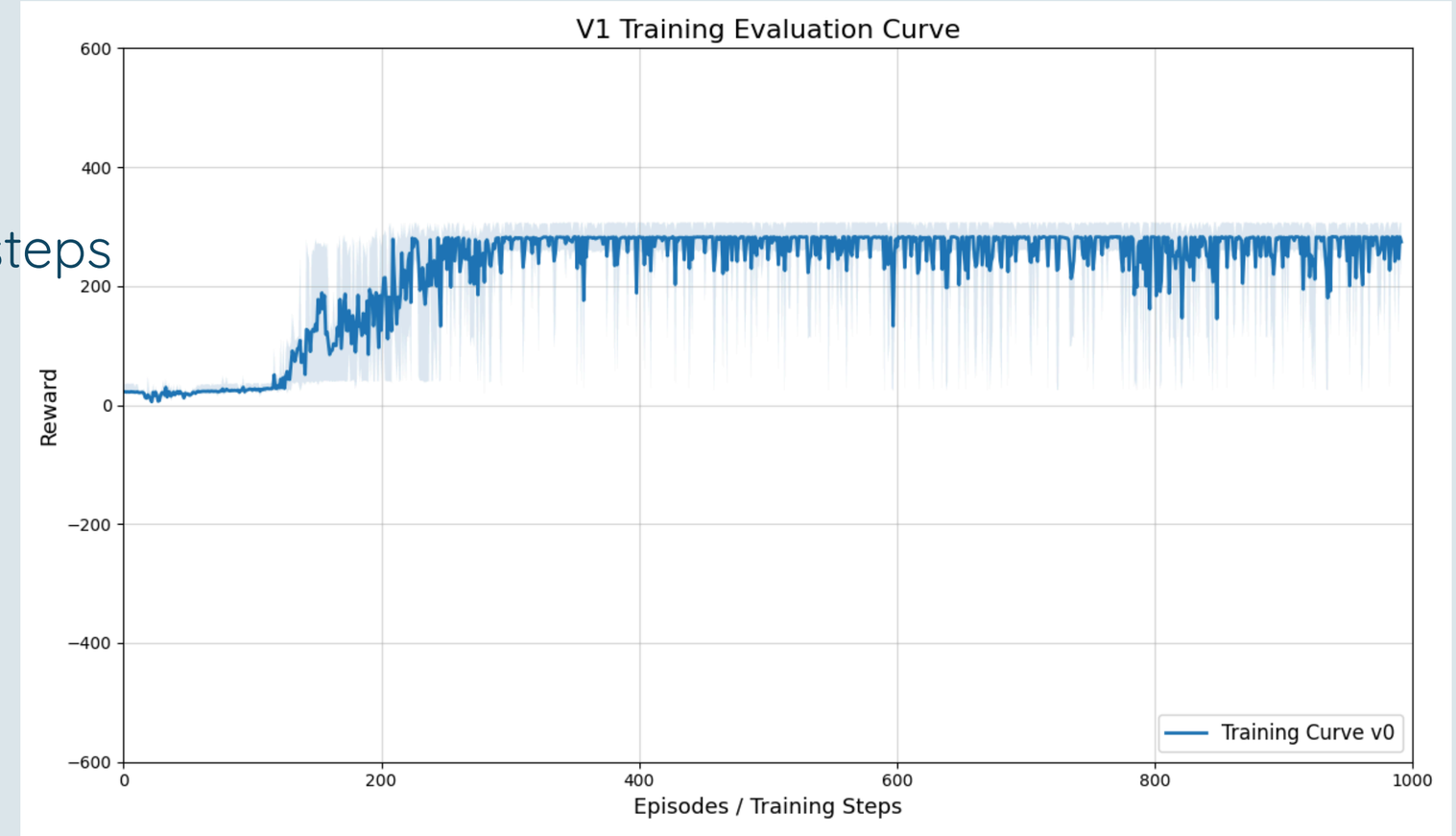
  continuous negative reward > 500 steps

→ done = True

- **Off-track Detection**

  all wheels leave the track

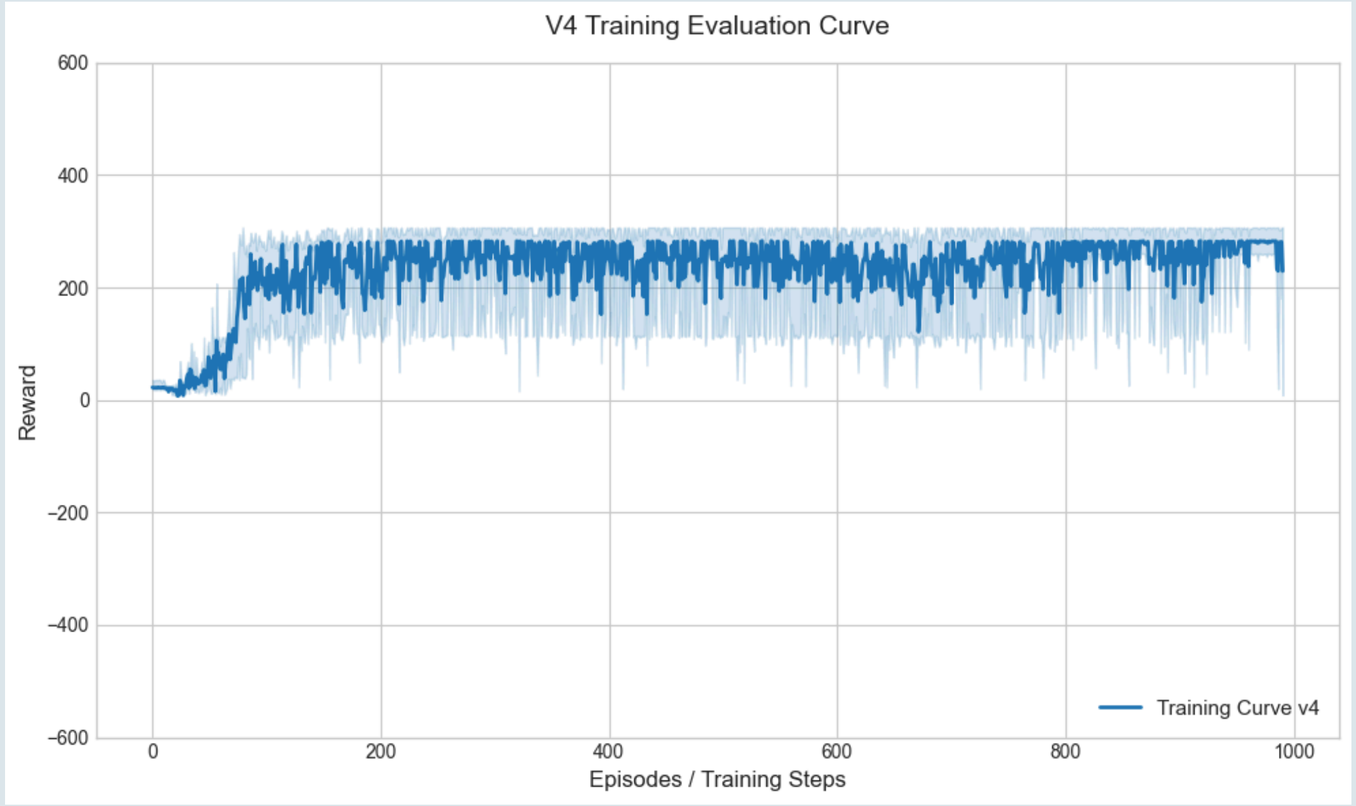
→ done = True



version 4

version 4: PER

- **Prioritized Experience Replay (PER)**
  Replaces Replay Buffer with PrioritizedReplayBuffer. Higher TD-error transitions → sampled more frequently



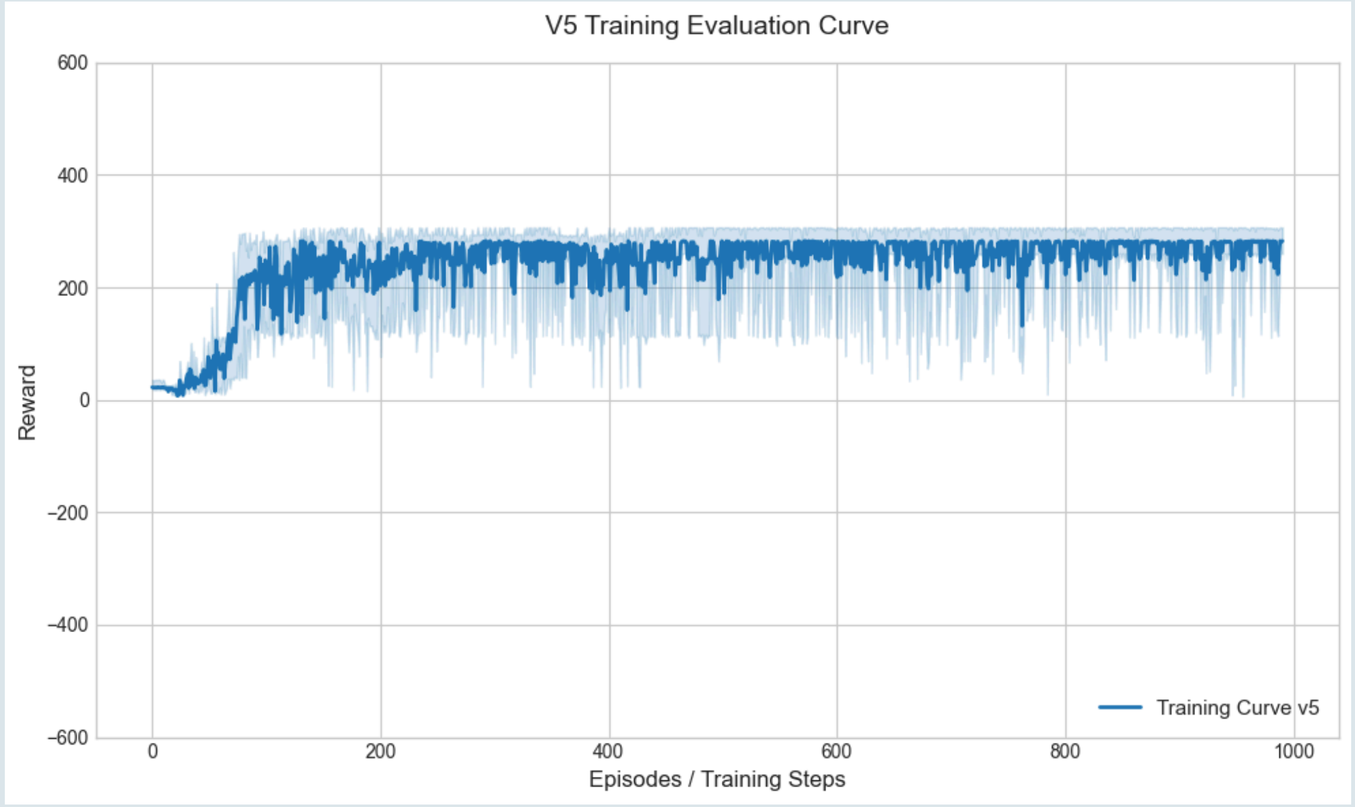
version 5

version 5: PER switched to ReplayBuffer after

Difference from V4:

- **Buffer Switching**

  Starts with PER, once best score exceeds 200 → migrate all data to Replay Buffer and continue with uniform sampling



Comparison and Conclusion

version 0 vs version 1

version 0: vanilla baseline that allows the car to drive unconstrained

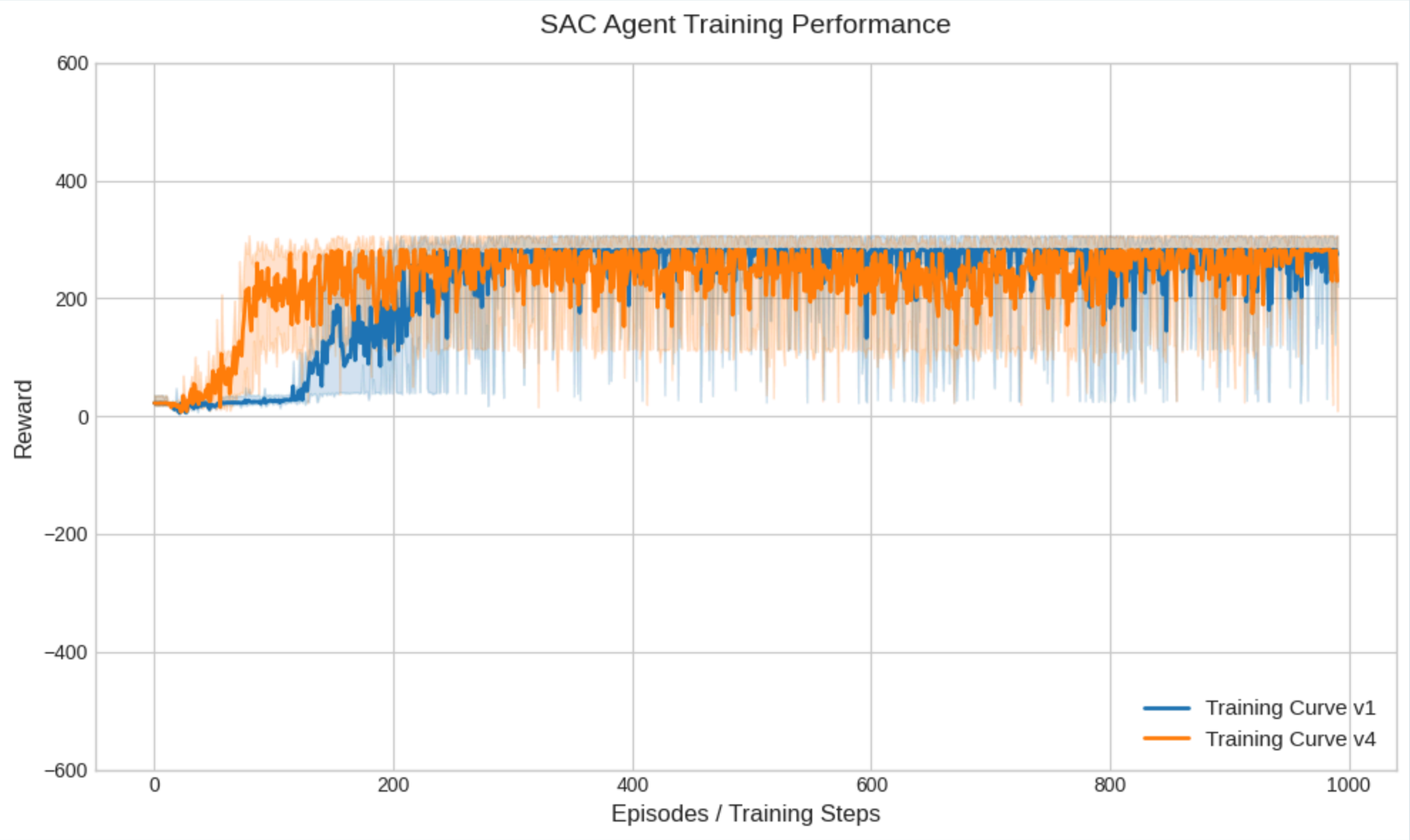
version 1: version 1(stuck-preventing, outside-preventing environment)



version 1 vs version 4

version 1: version 1(stuck-preventing, outside-preventing environment)

version 4: version 1(stuck-preventing, outside-preventing environment) + PER



version 4 vs version 5

version 4: version 1(stuck-preventing, outside-preventing environment) + PER

version 5: PER before best score exceed 200, then switch to version 1(stuck-preventing, outside-preventing environment)



Conclusion

v0->v1:修正v0因缺乏提早停止機制，導致模型停滯產生大量無效的數據。

v1->v4:解決隨機抽樣收斂過慢的問題，加上PER，利用TD-Error鼓勵模型複習關鍵錯誤，收斂時間提前了一倍

v4->v5:解決PER在訓練後期，因過於專注在特殊樣本而產生Overfiiting，讓後期的training performace變得比較穩定。

Thank you